

EE-2003

Computer Organization and Assembly Language

Week 07 – Lecture 13

Department of Computer Science/Software Engineering/ Artificial Intelligence
FAST-NUCES Peshawar

Outline

- 1 Quick Review
- 2 Bit Manipulations: Multiplication Algorithm
- 3 Shift Logical Right (SHR)
- 4 Shift Logical Left (SHL) / Shift Arithmetic Left (SAL)
- 5 Shift Arithmetic Right (SAR)
- 6 Rotate Left (ROL)
- 7 Rotate Through Carry Right (RCR)
- 8 Rotate Through Carry Left (RCL)
- 9 Comparison of Shift and Rotate Instructions
- 10 Shift and Rotate Instructions – Uses

Quick Review: What was Done Before Sessional - I?

- Conditional Jumps
- UnConditional Jumps
- Relative Addressing
- Types of Jump
- Bubble Sort

Bit Manipulations: Multiplication Algorithm

- With decision-making in place, we now study an algorithm for bit manipulations.
- Multiplication is familiar from early schooling, but we never saw it as an algorithm.
- Example: Multiply two 4-bit binary numbers:

$$1101 (13) \times 0101 (5) = 01000001 (65)$$

- Note: The answer is twice as long as the multiplier and multiplicand.

Step-by-Step Multiplication

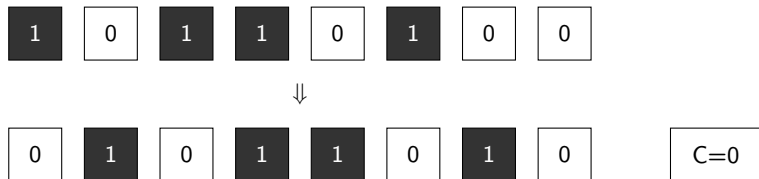
- Take the first digit of multiplier: if it is 1, write multiplicand; if 0, write 0.
- Each next line is shifted left by one position.
- Example:

$$\begin{array}{r} 1101 \\ 0101 \\ \hline 1101 \\ 0000x \\ 1101xx \\ 0000xxx \\ \hline 01000001 \end{array}$$

- In binary, digit-level multiplication is simple:
 - If $\text{digit} = 0 \rightarrow \text{result} = 0$
 - If $\text{digit} = 1 \rightarrow \text{result} = \text{multiplicand}$
- Each new digit in the multiplier shifts the result one more place left.
- Adding all intermediate results gives the final product.
- Next, we need shift instructions and a way to check multiplier bits one by one.

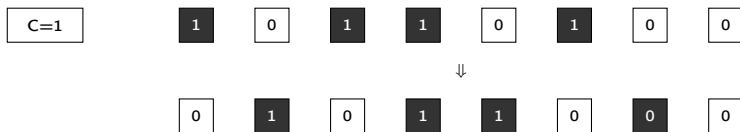
Shift Logical Right (SHR)

- Inserts a zero from the left and shifts all bits to the right.
- The rightmost bit moves into the **carry flag**.
- Example: $10110100 \xrightarrow{SHR} 01011010, C=0$
- For 16-bit registers, the same shift applies, but with 16 bits instead of 8.



Shift Logical Left (SHL) / Shift Arithmetic Left (SAL) – 8-bit

- Inserts a zero from the **right** and shifts all bits to the left.
- The most significant bit (MSB) moves into the **carry flag**.
- **SAL** is just another name for **SHL**.
- Example: $10110100 \xrightarrow{SHL} 01101000, C=1$



Shift Logical Left (SHL) / Shift Arithmetic Left (SAL) – 16-bit

- Same process applies to 16-bit registers (AX, BX, etc.).
- A zero enters from the right, all bits move left.
- The most significant bit (MSB) moves into the **carry flag**.
- Example: $1011010010110011 \xrightarrow{SHL} 0110100101100110, C=1$



Shift Arithmetic Right (SAR)

- For signed numbers, the most significant bit (MSB) holds the sign.
- In a logical right shift, inserting a zero from the left would change the sign.
- **SAR** shifts all bits to the right but keeps a copy of the MSB at the left.
- The least significant bit (LSB) is shifted into the **carry flag**.
- This preserves the sign while effectively dividing the signed number by 2.
- Left shift $\approx$ multiply by 2, right shift $\approx$ divide by 2.

Shift Arithmetic Right (SAR) – Example

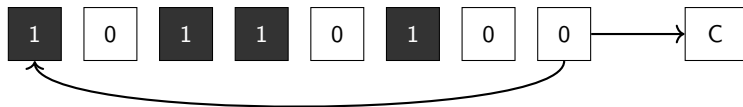


Example: $10110100 \xrightarrow{SAR} 11011010, C=0$

Rotate Right (ROR)

- In **ROR**, every bit moves one position to the right.
- The rightmost bit is copied into the **carry flag** and also reappears at the left.
- Imagine the pipe is circular: the last bit leaving the right end comes back into the left end.
- The carry flag records the bit that just rotated.

Rotate Right (ROR) – Example

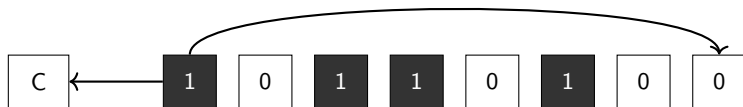


Example: $10110100 \xrightarrow{ROR} 01011010, C=0$

Rotate Left (ROL)

- In **ROL**, every bit moves one position to the left.
- The most significant bit (MSB) is copied into the **carry flag**.
- The same MSB also reappears on the right side of the register.
- This is the reverse of the **ROR** operation.
- Rotation can be performed on 8-bit or 16-bit registers.

Rotate Left (ROL) – Example

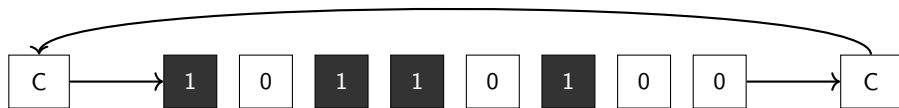


Example: 10110100 $\xrightarrow{ROL}$ 01101001, C=1

Rotate Through Carry Right (RCR)

- In **RCR**, the carry flag is inserted from the left.
- Every bit moves one position to the right.
- The rightmost bit is dropped into the **carry flag**.
- Effectively, this is a 9-bit rotation (8 bits + carry) or 17-bit rotation (16 bits + carry).
- Visualize the pipe as circular, but the carry flag is part of the circle.

Rotate Through Carry Right (RCR) – Example

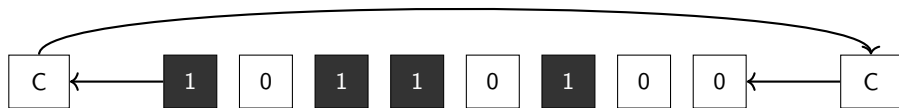


Example: $10110100 \xrightarrow{RCR} 01011010, C=0$

Rotate Through Carry Left (RCL)

- In **RCL**, the carry flag is inserted from the right.
- Every bit moves one position to the left.
- The most significant bit (MSB) moves into the **carry flag**.
- Effectively this is a 9-bit rotation (8 bits + carry) or a 17-bit rotation (16 bits + carry).
- It is the exact opposite of **RCR**.

Rotate Through Carry Left (RCL) – Example



Example: $10110100 \xrightarrow{RCL} 01101001, C=1$

Comparison of Shift and Rotate Instructions

Instruction	Operation
SHR	Insert 0 at MSB, shift right
SHL / SAL	Insert 0 at LSB, shift left
SAR	Copy MSB, shift right
ROR	Bits rotate right, LSB $\rightarrow$ MSB
ROL	Bits rotate left, MSB $\rightarrow$ LSB
RCR	Bits + CF rotate right
RCL	Bits + CF rotate left

Instruction	Carry Flag Effect
SHR	LSB $\rightarrow$ CF
SHL / SAL	MSB $\rightarrow$ CF
SAR	LSB $\rightarrow$ CF
ROR	LSB $\rightarrow$ CF
ROL	MSB $\rightarrow$ CF
RCR	LSB $\rightarrow$ CF, CF $\rightarrow$ MSB
RCL	MSB $\rightarrow$ CF, CF $\rightarrow$ LSB

Shift and Rotate Instructions – Uses

Instruction	Use / Purpose
SHR (Shift Logical Right)	Logical divide by 2 (unsigned numbers)
SHL / SAL (Shift Left)	Multiply by 2 (unsigned numbers)
SAR (Shift Arithmetic Right)	Arithmetic divide by 2 (signed numbers)
ROR (Rotate Right)	Bit scrambling, cryptography, checksums
ROL (Rotate Left)	Bit scrambling, cryptography, checksums
RCR (Rotate Through Carry Right)	Multi-word shifts, extended precision ops
RCL (Rotate Through Carry Left)	Multi-word shifts, extended precision ops

Shifts vs Rotates – Summary

Shifts (Arithmetic)	Use / Purpose
SHR ($\rightarrow$ Shift Logical Right)	Logical divide by 2 (unsigned numbers)
SHL / SAL ($\leftarrow$ Shift Left)	Multiply by 2 (unsigned numbers)
SAR ($\rightarrow$ Arithmetic Right)	Arithmetic divide by 2 (signed numbers)
Rotates (Bit Manipulation)	Use / Purpose
ROR ($\circlearrowright$ Rotate Right)	Bit scrambling, cryptography, checksums
ROL ($\circlearrowleft$ Rotate Left)	Bit scrambling, cryptography, checksums
RCR ($\circlearrowright$ +CF Rotate Through Carry Right)	Multi-word shifts, extended precision ops
RCL ($\circlearrowleft$ +CF Rotate Through Carry Left)	Multi-word shifts, extended precision ops

Examples: $\text{SHL}(5) = 10$, $\text{SHR}(8) = 4$

Note: Shifts are mainly used for arithmetic, while Rotates are used for bit-level manipulations.

Thank You!

Any Questions?

- We explored: **Shift and Rotate instructions** (SHR, SHL, SAR, ROR, ROL, RCR, RCL).
- Key lessons:
 - Shifts are mainly used for **arithmetic operations** (multiply/divide by 2).
 - Rotates are essential for **bit-level manipulations** (crypto, checksums, device control).
 - Carry Flag (CF) plays a crucial role in rotate-through-carry instructions.
 - Understanding signed vs unsigned shifts avoids logical errors.
- Takeaway: **Bit manipulations are powerful tools for efficient low-level programming.**

“Master shifts and rotates — and you master bit-level control.”